Fullstack Take-Home Assignment

Build a small fullstack app that **imports a bank statement (CSV), auto-categorizes the transactions, and shows the accountant a clean summary** — frontend in **Nuxt 3**, backend in **Python / FastAPI**.

This is a feature we do **not** have yet, so don't copy anything — build it from scratch the way you think is right.

> *You are expected to build this with AI (Cursor / Claude Code / Codex). That's how we work. We care about how you drive the AI and whether you understand the result — not whether you type code by hand.*

Why This Feature

Accountants drown in bank statements. Every month they get a CSV from the bank with hundreds of transactions and have to manually sort each one into a category (rent, salary, taxes, suppliers, fees, etc.). We want a tool that does the boring part automatically.

What to Build

1. Import

- Upload a **CSV bank statement** and parse it into clean transactions (date, description, amount, currency).
- The sample file is **sample_statement.csv** — build against it. It is a realistic, messy export:
- Has **header/metadata junk lines** at the top before the real table starts.
- Money is split into **two columns** (Debit and Credit) instead of one signed amount — you decide the sign.
- **Dates** are in DD.MM.YYYY, **amounts** use a comma as decimal separator and spaces as thousands separators (e.g. 1 234,56), and there are **multiple currencies**.
- There are **genuinely broken rows** (missing fields, garbage values, a duplicate) mixed in.
- Parse it robustly: normalize the data, and **skip/flag broken rows instead of crashing**. Surface to the user how many rows were imported vs. skipped.

2. Auto-Categorize

- Each transaction gets a **category** automatically based on **rules** the user defines.
- A rule = "if the description contains X → category Y" (e.g. contains SALARY → Payroll, contains MIA / Orange → Telecom).
- The user can **add / edit / delete rules**, and re-running categorization re-applies them.
- Anything that matches no rule lands in **Uncategorized**, and the user can assign a category by hand (which can optionally become a new rule).

3. Summary Screen

- A table of all transactions with their category, plus filters (by category, by month) and search (by description).
- A summary on top: **total income, total spending, balance, and a breakdown per category** (a simple chart is a plus, not required).

> *The three parts must connect: importing feeds the table, editing rules changes the categories, the summary reflects whatever the rules produced.*

Stack (Non-Negotiable)

Backend

Python + **FastAPI** + **Pydantic v2**, data stored via **SQLAlchemy** (SQLite is fine). Keep business logic out of the route handlers — the parsing and the categorization engine should live in their own service so they're easy to reason about.

Frontend

Nuxt 3 + Vue 3 + TypeScript, Composition API. Tailwind or UnoCSS for styling. At least one composable for shared logic. Handle loading / empty / error states. It should look clean and be obvious to use — we don't have a designer, so good UX is on you.

> *Nuxt is our main frontend and you said you only know it "a little". That's intentional — show us you ramp up fast with AI. Honest working Nuxt 3 beats a React app.*

How to Work with AI (This Matters)

- **Plan first**: have the AI produce a plan, you review and edit it, then execute. Keep the plan/spec in the repo.
- **Decompose** the work; use subagents where it helps.
- Open at least **one Pull Request** and run it through an **AI code reviewer** (Devin / CodeRabbit / Cursor Bugbot — whatever you use) and address the comments. Leave it visible.
- Show us at least **one place where the AI was wrong** and you overruled it — and why.

Deliverables

1. A **public GitHub repo** (one repo, two folders for back/front is fine).
2. A clean **git history** — small, meaningful commits, not one giant dump.
3. At least one **PR with AI review** visible.
4. A short **README**: how to run both apps in a few minutes, a quick architecture overview, and a section **"How I built this with AI"** (tools, how you split it, what you kept vs. rejected).
5. Send the repo link when done.

Scope

Keep it focused. Don't gold-plate. A correct, well-explained **import** → **categorize** → **summary** flow that you can reason about beats a sprawling half-working app. **Auth, deployment and CI are not needed** — local run is fine.